

Clash of Codes

Real-Time Competitive Coding & Strategy Game

Game Design Document (GDD)

Core Architecture & Gameplay Specifications

2026

Abstract

Clash of Codes is a multiplayer competitive programming game that adapts the high-stakes, real-time resource management of the *Clash Royale* format into an algorithmic battleground. Players match with opponents globally or challenge friends in private rooms, balancing fast-paced code composition, multiple-choice questions, and disruptive environmental abilities called Spells. This document outlines the fundamental loop, economic resource models, combat mechanics, and matchmaking criteria.

Contents

1	The Core Game Loop & Resource Economy	2
1.1	Battle Elixir (BE)	2
1.2	Rank Points (RP) and the Conversion Mechanic	2
2	Match Structure & Victory Conditions	2
2.1	The Three-Track Node Setup	2
2.2	Node Workflow Structure	3
3	The Spellbook Mechanics (Attacks & Defenses)	3
3.1	Offensive Spells (Attacks)	3
3.2	Defensive Spells (Counter-Play & Fortification)	4
4	Matchmaking Models & Environments	4
4.1	Ranked Ladder (Random Mode)	4
4.2	Custom Lobbies & Private Architecture	4
4.3	Syntax Stages (Arenas)	4

1 The Core Game Loop & Resource Economy

The gameplay ecosystem transforms isolated technical challenges into a fluid strategy loop. The engine tracks two distinct resources to measure performance and execute moves: **Battle Elixir (BE)** and **Rank Points (RP)**.

1.1 Battle Elixir (BE)

Battle Elixir is the volatile, real-time resource used exclusively *during* an active match to deploy tactical spells.

- **Capacity Limits:** All players initiate a match with 0 BE. The maximum capacity pool is hard-capped at 10 BE.
- **Passive Generation:** During standard play, players regenerate 1 BE every 10 seconds. Upon entering the final 60 seconds of a match, the engine enters **Double Elixir** mode, accelerating passive replenishment to 2 BE every 10 seconds.
- **Active Generation:** Progression milestones award active bonuses to reward efficient completion:

Successful Compilation Milestone $\rightarrow +1$ BE

Flawless First-Submission Run $\rightarrow +3$ BE

Correct MCQ Answer Entry $\rightarrow +1$ BE

1.2 Rank Points (RP) and the Conversion Mechanic

Rank Points dictate a user's global ladder standing. While victory or defeat awards or subtracts flat RP baselines, the game implements a unique **Greed vs. Power Economy**:

$$\text{Final RP Awarded} = \text{Match Base Victory RP} + (\text{Unspent BE Remaining at Match End} \times \delta) \quad (1)$$

Where δ represents the static scaling multiplier conversion factor. This introduces a critical tactical choice: players must choose between burning Elixir on aggressive spells to guarantee a win, or conserving Elixir at the risk of losing to maximize their net ranking gains upon victory.

2 Match Structure & Victory Conditions

Matches occur on a dynamic, split-screen UI displaying the player's active Integrated Development Environment (IDE) alongside a real-time progress tracker of the opponent's test case passing rates.

2.1 The Three-Track Node Setup

A standard map consists of three separate algorithmic tracks, serving as the equivalent of battlefield towers:

1. **Left Track (Easy Complexity):** Yields 1 Crown upon complete resolution.
2. **Right Track (Medium Complexity):** Yields 1 Crown upon complete resolution.
3. **Center Track (Hard Complexity):** Represents the **King's Tower**. Fully resolving this node instantly triggers a 3-Crown absolute victory, overriding all remaining match parameters.

2.2 Node Workflow Structure

Every individual Problem Node follows a strict two-stage sequential architecture:

Stage I: Code Composition (Pass Public Test Cases) → Stage II: Core MCQ Stream (3–5 Questions)

Point deduction mechanics are mapped to error conditions: wrong code submissions deduct flat pool values, and incorrect MCQ responses inflict immediate negative point scaling penalties.

3 The Spellbook Mechanics (Attacks & Defenses)

Before entering matchmaking queues, players assemble a customized loadout containing exactly ****4 Spells**** (configured as 2 Offensive and 2 Defensive choices).

3.1 Offensive Spells (Attacks)

Offensive abilities are designed to disrupt the opponent’s focus, corrupt their interface, or alter their programming environment.

Table 1: Offensive Spell Registry

Spell Name	BE Cost	Mechanical & System Impact
Fog of War	2	Obfuscates compiler responses for 45s. The opponent only receives generic "Compilation Failed" messages without line designations.
Time Warp	3	Distorts the opponent’s match timer rendering, forcing the clock UI to spin visually at double speed for 30s to trigger panic.
Chaff	4	Blackouts and obfuscates 50% of the sample input and output verification values on the opponent’s interface.
EMP (Flicker)	4	Causes the line numbers in the opponent’s code editor to scramble and causes the screen to flash dynamically for 15s.
Syntax Poison	4	Switches the opponent’s active IDE theme to an unreadable, ultra-low contrast color scheme for a duration of 30s.
MCQ Barrage	5	Force-injects 2 high-difficulty, conceptually complex multiple-choice items directly into the opponent’s validation queue.
The Typo Curse	5	For 30s, whenever the opponent inputs a standard semicolon (;), there is a 30% chance it maps to a Greek question mark (̸) breaking compiling state.
Bracket Bandit	6	Evaluates the active buffer of the opponent’s editor and instantly deletes all trailing closing structural symbols (},],).
Memory Leak	6	Caps the absolute execution environment runtime boundary for 45s, causing valid code to fail due to artificial Time Limit Exceeded (TLE).
Code Swap	8	Mutates the opponent’s targeted track challenge into a completely new algorithmic problem of equivalent tier, resetting progress.

3.2 Defensive Spells (Counter-Play & Fortification)

Defensive spells allow players to neutralize active curses, reveal hidden metrics, or secure their progress tracks.

Table 2: Defensive Spell Registry

Spell Name	BE Cost	Mechanical & System Impact
Clock Sync	1	Dispels all active clock-spinning illusions, instantly re-aligning the local timer interface with the true server timestamp.
Lint Shield	2	Deploys an execution wrapper rendering the local IDE completely immune to screen flickering, palette shifts, or typo curses for 60s.
Debugger Sight	3	Evaluates local runtime structures and permanently surfaces 2 hidden edge-case inputs for the active node challenge.
Sanitize Input	3	Instantly purges active hidden test parameters or UI obfuscations caused by incoming Chaff spells.
Ctrl + Z (Rollback)	4	Activates a short reactive window (3s); if struck by a Code Swap, this rolls back the mutation and restores the previous code buffer.
Firewall	4	Encapsulates a selected Problem Track. For the next 90s, that specific objective node cannot be altered, swapped, or sabotaged.
Garbage Collector	5	Forces a total system flush, purging all active debuffs, curses, visual distortions, and status impairments from the player's UI.

4 Matchmaking Models & Environments

The system establishes balance through segmented queue pathways matching users by skill, goals, and language preference.

4.1 Ranked Ladder (Random Mode)

The public competitive matchmaking algorithm pairs components using an optimized Elo matching standard based on current global Rank Points. The system selects questions randomly across diverse sub-disciplines (e.g., Arrays, Dynamic Programming, Graph Theory) tailored to the match's language selection.

4.2 Custom Lobbies & Private Architecture

Users can bypass public queues to construct custom sandbox parameters using structural entry points:

- **Room Codes:** Generates an alphanumeric key allowing immediate peer connection.
- **Direct Invites:** Sends targeted push notifications utilizing a player's distinct User ID.
- **Configuration Controls:** Room hosts can alter game conditions by filtering topics, capping node difficulty bounds, or altering Elixir regeneration scaling (e.g., Infinite 7x Elixir modes).

4.3 Syntax Stages (Arenas)

As users cross key RP thresholds, they ascend through visual and algorithmic arenas:

- **Arena 1: Spaghetti Junction (0 – 999 RP):** Focused on primitive operations, array traversals, and basic string parsing.
- **Arena 5: The Stack Overflow (2000 – 2999 RP):** Introduces recursive trees, heavy graph algorithms, and custom object mapping.
- **Arena 10: Kernel Palace (5000+ RP):** Features advanced systems architecture optimizations and multi-variable dynamic scheduling challenges.